

*Chapter No 5 – Introduction to Turbo C Programming Environment***CE – 101 : C & P F***Computing and Programming
Fundamentals***Batch 2015**

Sir Syed University of Engineering & Technology
Computer Engineering Department
University Road, Karachi-75300, PAKISTAN

*Compiled By:**Syed Atir Iftikhar***CE - 101 : Computing and Programming
Fundamentals****■ Course Objectives:**

- Upon successful completion of this course, the student will be able to:
 - Fundamentals of Computer Engineering and Information Technology
 - Overview of History, Classification and Components of Computers
 - Number Systems and Logic Gates
 - Overview of Software, Operating Systems, Networks and Internet
 - Introduction about Programming
 - Basic Building Blocks, Loop, Decision Making

CPF Books

▪ Text Book:

1. Introduction to Computers (7th Edition)

By Peter Norton

2. Turbo C Programming For The PC (Revised Edition)

By Robert Lafore

▪ Reference Books:

1. Computer, Communications and Information.

By Sarah Hutchinson and Stacey Sawyer

2. Let Us C

By Yashavant Kanetkar

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

2
0
1
4
3

Marks Distribution

▪ Mid Term _____ 20

▪ Lab Work + Project _____ 20

▪ Quiz _____ 5

▪ Assignment + Class Performance _____ 5

▪ Semester Final Paper _____ 50

▪ Total Marks _____ 100

▪ *Performance Bonus* _____ *5 Marks*

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

2
0
1
4
4

2

Course Instructors

- **Syed Aatir Iftikhar**

satirs@hotmail.com

Lecturer, CED

Room No: BF-03

2
0
1
4

7

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

Course Outline

✓ PART A

- History of Computer
- Classification of Computer
- Basic Components
- CPU, I/O, Peripheral Devices, Storage
- Von Neumann Architecture
- Number Systems
- Binary Numbers
- Boolean Logic
- Software
- Operating Systems
- Networks and Internet

✓ PART B

- Types of Programming Languages
- Algorithm
- Flow Chart
- Introduction to C Language Programming
- Basic Building Blocks
- Loops
- Decision Making Statements

2
0
1
4

8

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

4

CE – 101: Computing and Programming Fundamentals

Chapter**5****Batch 2014**

Introduction to Turbo C Programming Environment

Chapter 5 - Contents

- Algorithm and Pseudo code
- Flow Chart
- Computer Language
- Types of Computer Languages
- Processing a High Level Language Program
- Introduction to C Language
- Why use C Language?
- Turbo C Development System
- Files used in C Program Development
- Writing and Saving a Program in C Language
- The Basic Structure of C Program

2**0****1****4****10**

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

5

Computer Programming

- Computers do “*what we tell them to do*”, NOT “*what we want them to do*”.
- **Computer Programming** involves writing instructions and giving them to the computer to complete a task.

2
0
1
4

11

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

Algorithm and Pseudo code

- An **Algorithm** is sequence of steps, by which a computer can produce the required outputs from the available inputs.
- “*An effective procedure for solving a class of problems in a finite number of steps.*”
- **Pseudo code** is an artificial and informal language that helps programmers develop algorithms. Pseudo code is very similar to everyday English.

2
0
1
4

12

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

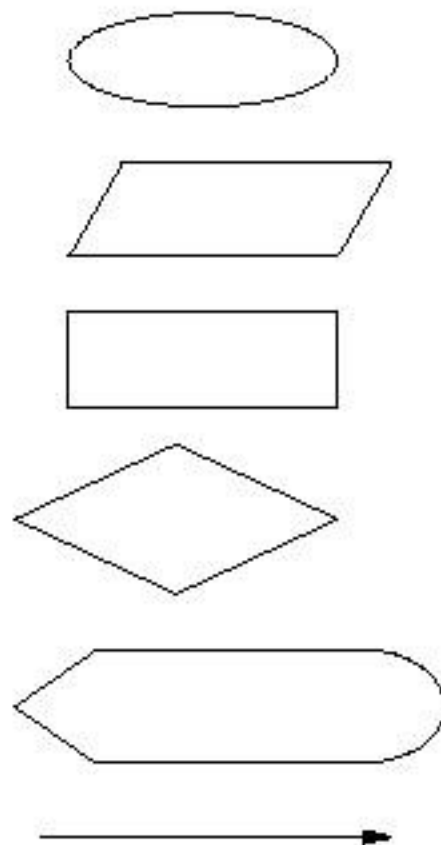
6

Flow Chart

- A Flowchart is a visual representation of an algorithm
- A Flowchart uses easy-to-understand symbols to represent actions on data and the flow of data
- Flowcharts aid in breaking down a problem into simple steps

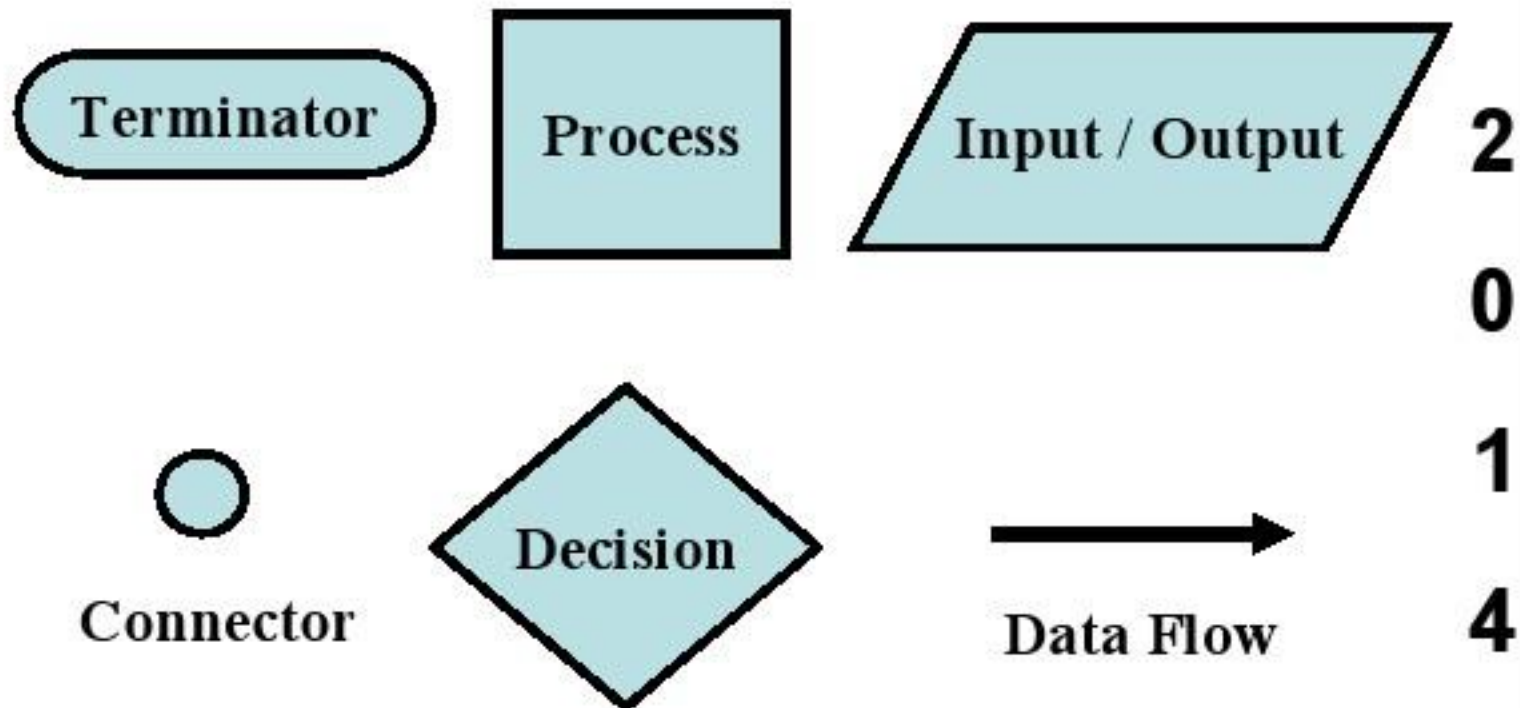
2014

Flow Chart



2014

Flow Chart



CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

15

Algorithm & Pseudo code [Example 1]

- Write an algorithm to determine a student's final grade and indicate whether it is passing or failing. The final grade is calculated as the average of four marks.

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

16

Algorithm & Pseudo code [Example 1]**Pseudo code:**

- *Input a set of 4 marks*
- *Calculate their average by summing and dividing by 4*
- *if average is below 50*

Print "FAIL"

else

Print "PASS"

2
0
1
4

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

17

Algorithm & Pseudo code [Example 1]▪ **Algorithm**

- Step 1: Input M1,M2,M3,M4
- Step 2: $\text{GRADE} \leftarrow (\text{M1} + \text{M2} + \text{M3} + \text{M4}) / 4$
- Step 3: if (GRADE < 50) then
 - Print "FAIL"
 - else
 - Print "PASS"
- endif

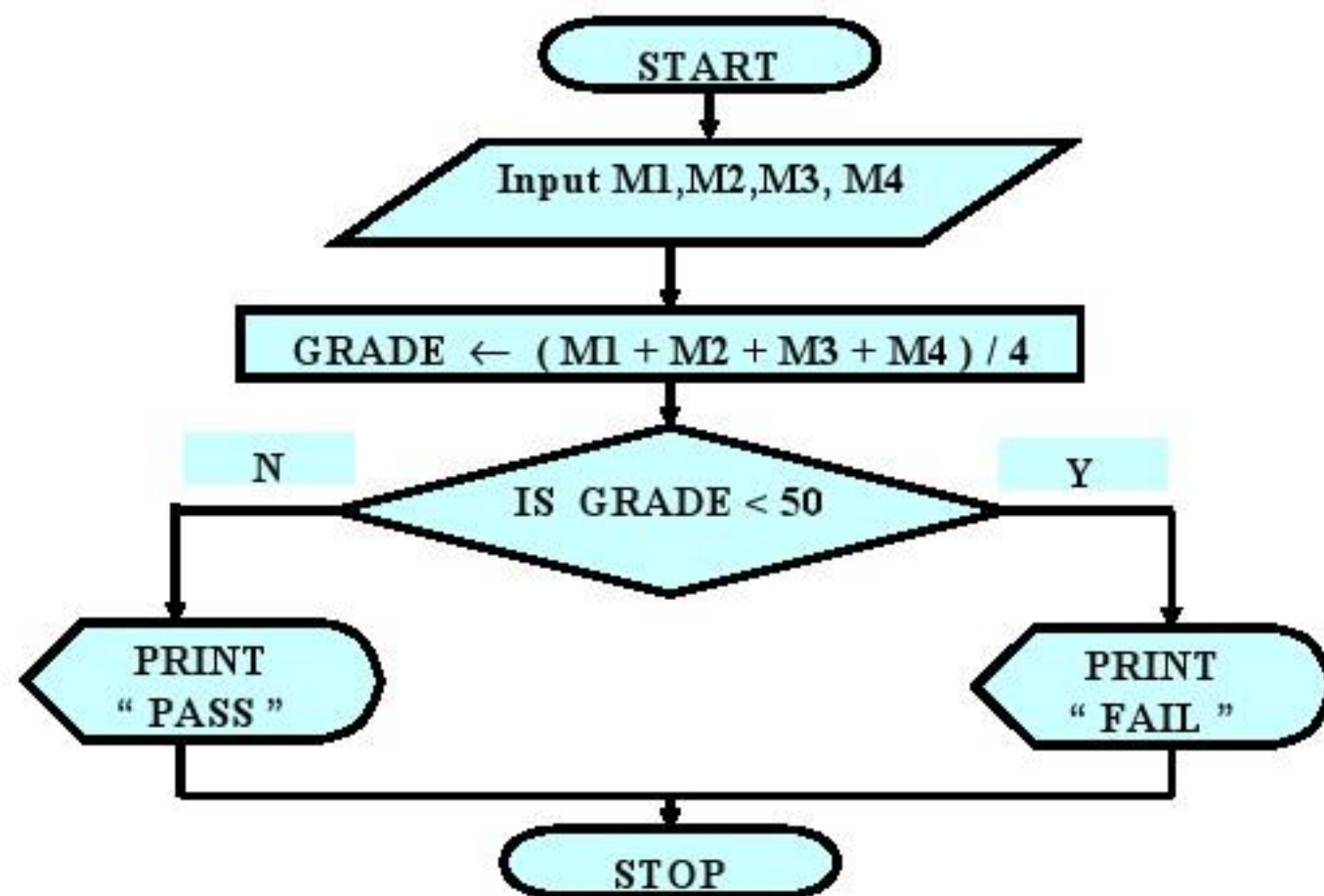
2
0
1
4

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

18

9

Algorithm & Pseudo code [Example 1]



CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

19

Algorithm & Pseudo code [Example 2]

- Write an algorithm and draw a flowchart that will read the two sides of a rectangle and calculate its area.

Pseudo code

- Input the width (W) and Length (L) of a rectangle*
- Calculate the area (A) by multiplying L with W*
- Print A*

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

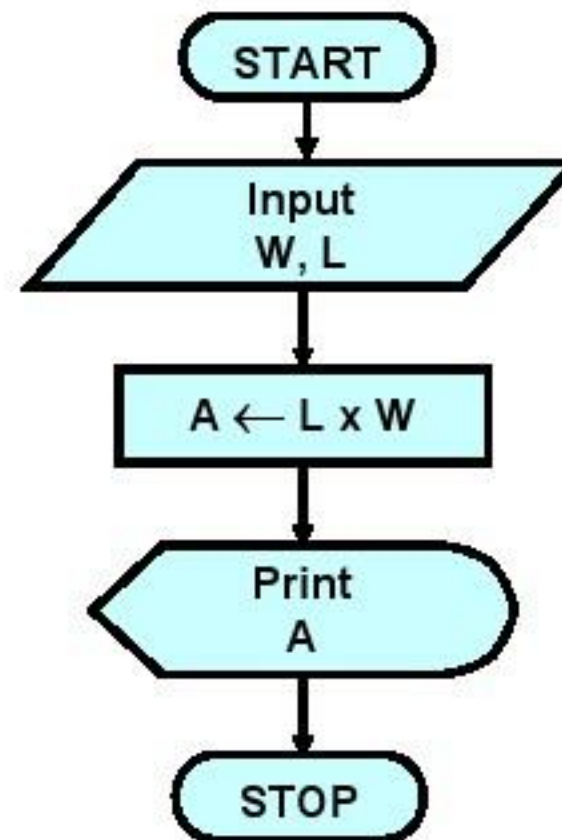
20

10

Algorithm & Pseudo code [Example 2]

Algorithm

- Step 1: Input W,L
- Step 2: $A \leftarrow L \times W$
- Step 3: Print A

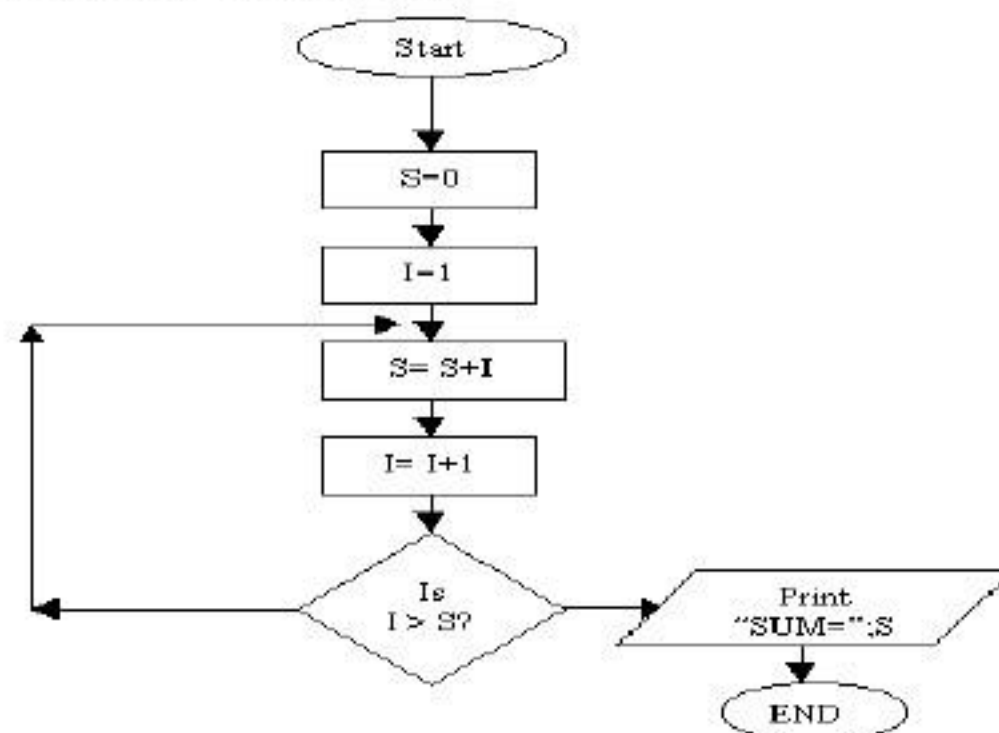


CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

201421

Example of Flow Chart

- Flowchart for finding the sum of first five natural numbers (i.e. 1,2,3,4,5):



CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

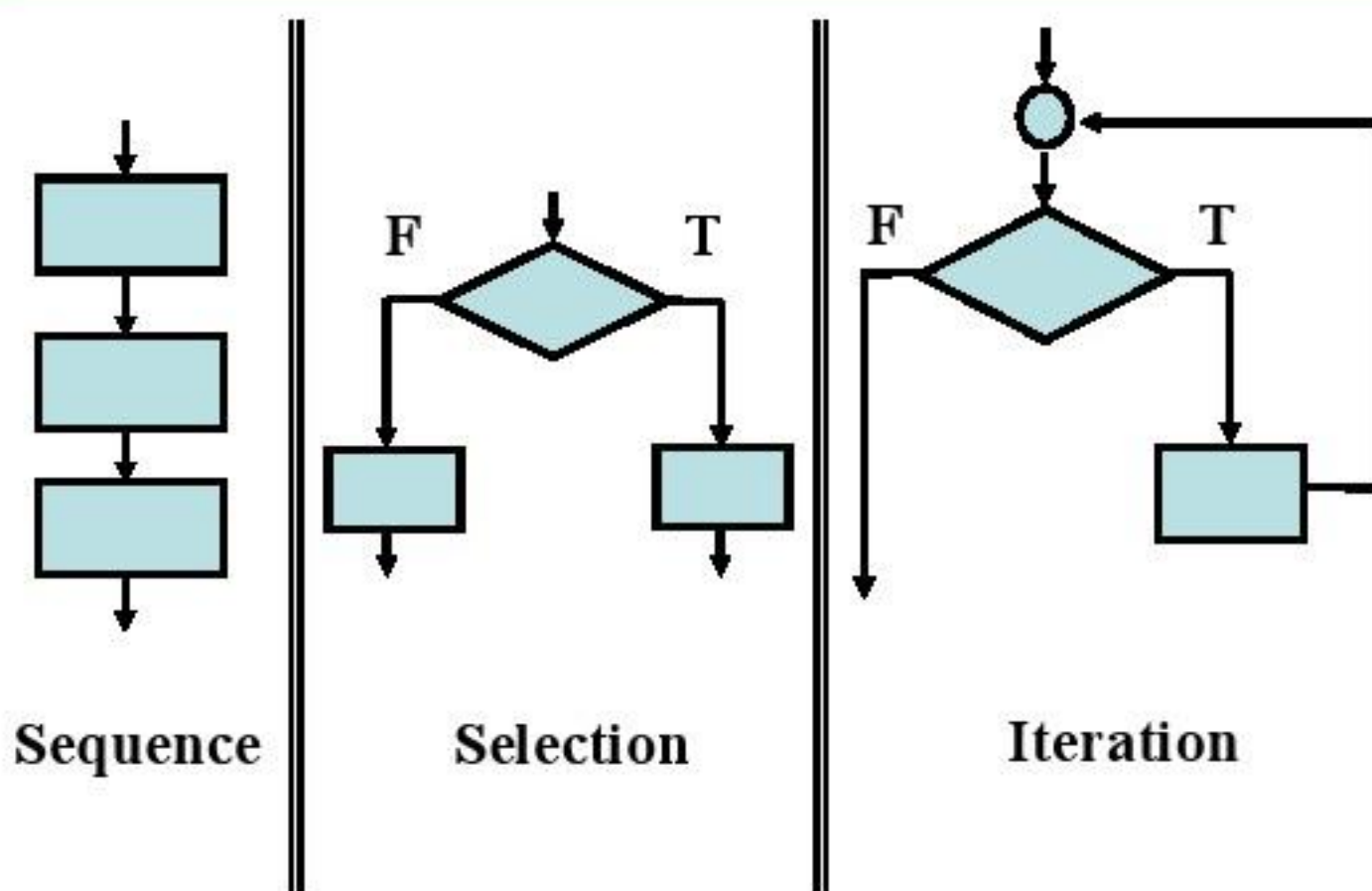
201422

Three Computer Programming Constructs

- Any problem, regardless of how complex, can be broken down into three basic *CONSTRUCTS*
 - SEQUENCE
 - SELECTION
 - ITERATION

2014

Three Computer Programming Constructs



2014

Computer Language

- A computer language is a set of rules and conventions used to convey the information to a computer

- **3 Types of Computer Languages**

1. Machine Language
2. Low Level Language
3. High Level Language

2014

3 Types of Computer Languages

- **Machine Language**

- The native tongue of a computer is the Machine Language. Each Machine Language instruction is a binary string of 0's and 1's that specifies an operation and identifies the memory cells involved in that operation.

2014

3 Types of Computer Languages

- **Low Level Language**

- In the Low Level Language, Machine Language is still used by the computer as it processes data, but Low Level Language software first translate the specified operation symbol onto its Machine Language equivalent.
- Example is Assembly Language.

2014

3 Types of Computer Languages

- **High Level Language**

- High Level Language is a programming language where an instruction resembles everyday language. Instructions are given to a computer by using a convenient letters, symbols or English text rather than by using 1's and 0's code that the computer understands.
- Example is Basic and Pascal etc.

2014

Difference between the High Level Language and the other two

- The main difference between the High Level Language and the other two are that High Level Language is much easier and understandable by a human being and the second difference is that in High Level Language one instruction can perform several machine level instructions.

2014

Where C Language Stands

- C Language in between the Low Level Language and High Level Language. That's why it is also called a *Middle Level Language*, since it was designed to have both: a relatively good programming efficiency and relatively good machine efficiency.

2014

Relation between High Level Language and Machine Language

- Because a computer understands only programs written in machine language, each instruction in the High Level Language programs must first be translated into the machine language before it can be executed.
- The original High Level Language written program is called the **Source Program**; and the machine language translation is called the **Object Program**.

2014

31

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

Processing a High Level Language program

1. Use an editor or a word processor program to enter each line of the source program into memory and to save it on disk as a source file.
2. Use a compiler program to translate the source program into machine language. If there are syntax error i.e error in grammar, the compiler displays these errors on the monitor. Use the editor program to correct the errors by editing and resaving the source program.

2014

32

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

16

Processing a High Level Language program

3. When the source program is free of syntax errors, the compiler saves its machine language translation as an object program.
4. The linker program combines your object program with additional object files that may be needed for your program to execute and saves the final machine language program an executable file on the disk.
5. The loader program places the executable file in the memory, ready for execution.

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

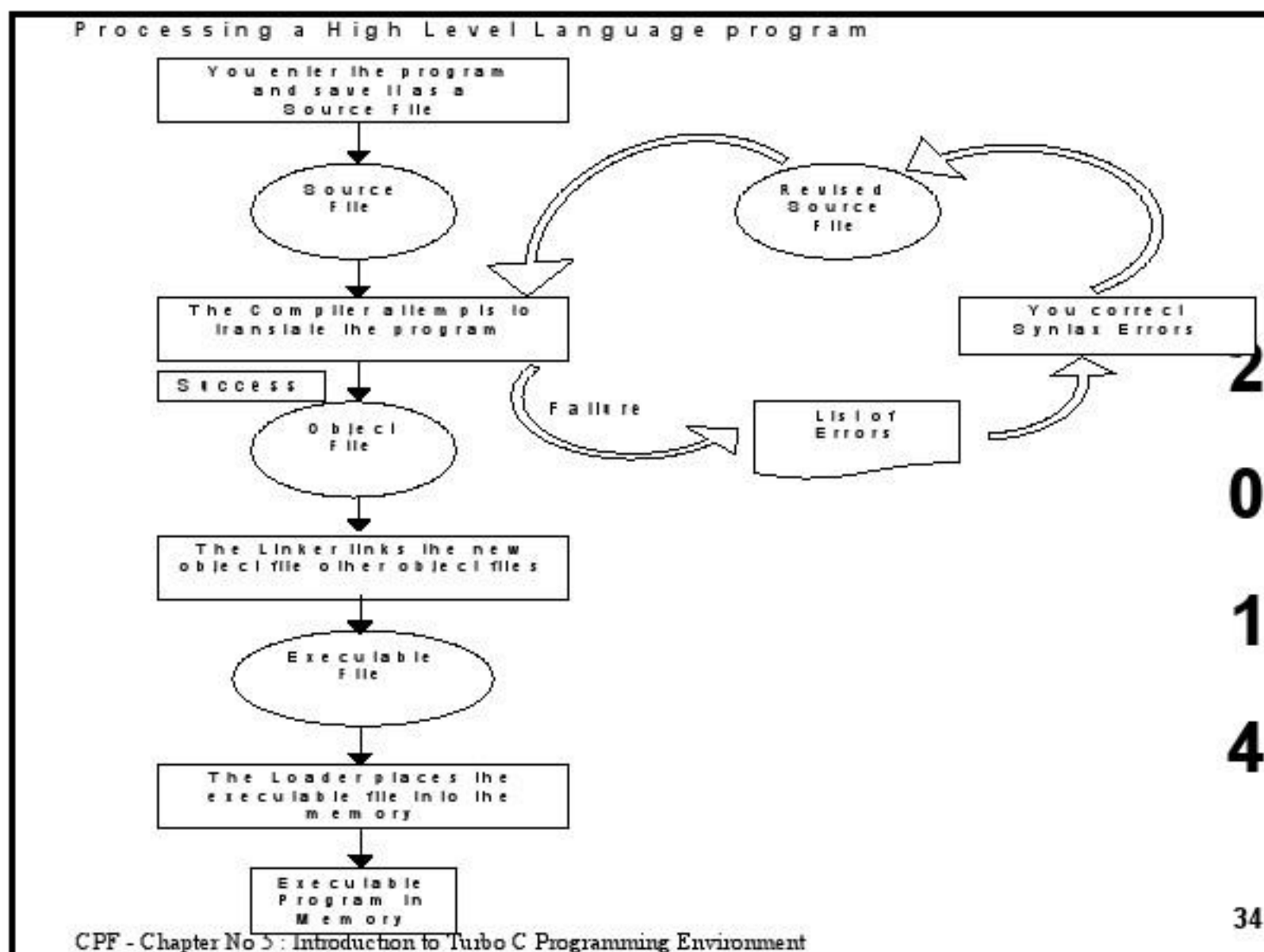
2

0

1

4

33



2

0

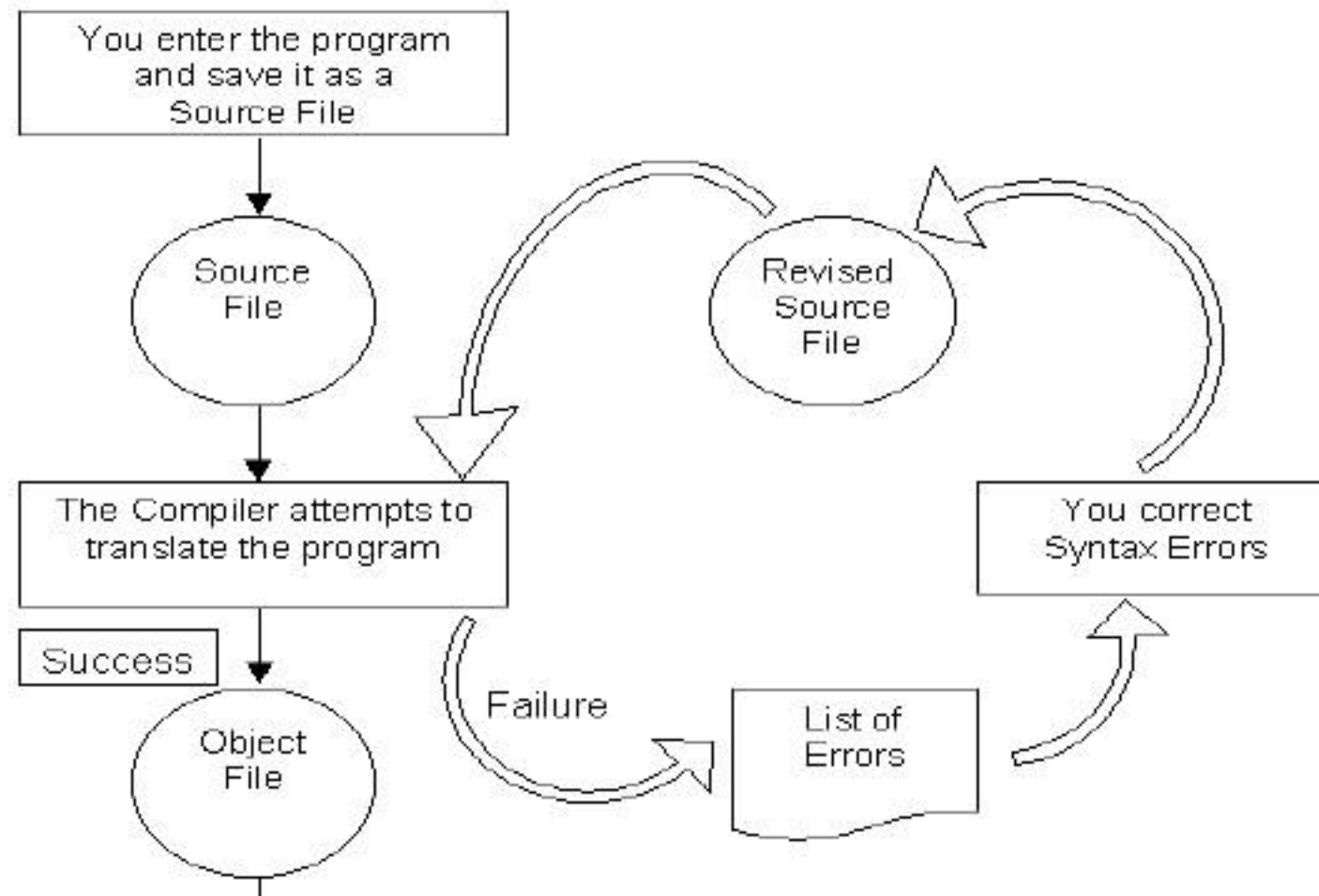
1

4

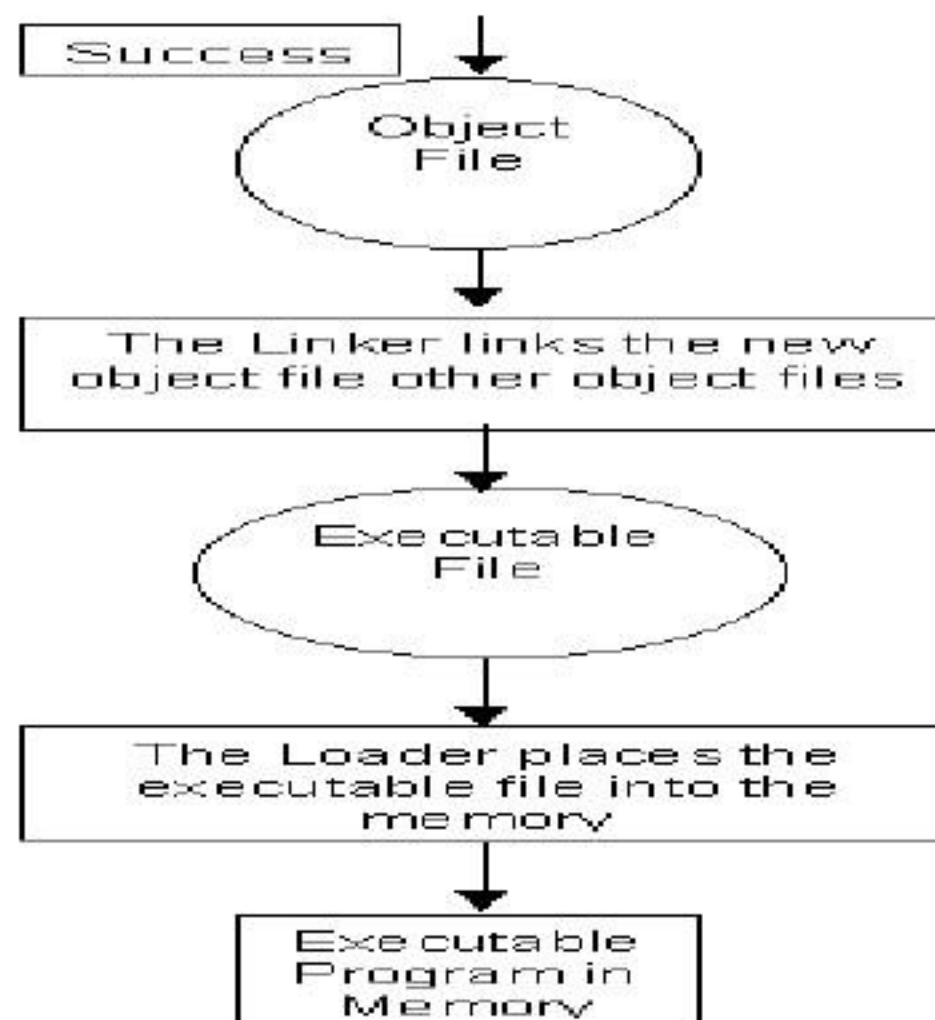
34

17

Processing a High Level Language program



CPF - Chapter No 5 : Introduction to Turbo C Programming Environment



CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

2
0
1
4

36

18

Introduction to C Language

- C, a high-level language programming language was developed in early 1970s by Dennis Ritchie at Bell Laboratories. Over the years, the power and flexibility of C, together with the availability of high quality C compilers for computers of all sizes, have made it a popular language in industry for a wide variety of applications.

2014

Why use C Language?

1. C is the most popular PC programming language.
2. C is unique in programming language in that it provides the convenience of a high level language such as Basic or Pascal, but at the same time allows much closer control of a computer's hardware and peripherals, as the assembly language does.
3. C compilers can generates amazingly fast code.

2014

Why use C Language?

4. C Language is a well-structured and modular language. Its syntax makes it easy to write programs that are modular and therefore they are easy to understand and maintain.
5. C Language is portable: i.e it is easier to convert a C program to run on a different machine than it is to convert programs written in most other languages.
6. C Language IDE is more user-friendly than most other languages.

2
0
1
4

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

39

Turbo C Development System

There are two development system in Turbo C

1. IDE
2. Command-Line Development System

2
0
1
4

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

40

20

Turbo C Development System

▪ IDE (Integrated Development Environment)

- It is a screen display with windows and pull down menus. The program listing, its output, error messages and other information are displayed in separate windows.
- You can use menu selections to invoke all the operations necessary to develop your program including editing, compiling, debugging, linking and program execution.

2014

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

41

Turbo C Development System

▪ Command-Line Development System

- This is a traditional command line system, in which editing, compiling, debugging, linking and program execution are invoked from the DOS command line prompt as separate activities, performed by separate programs.

2014

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

42

21

Memory Models

- There are five memory models:
 - *Small, Compact, Medium, Large and Huge.*
- In the Small model your program can have upto 64K of code and 64K of data storage.
- In the Medium model the program code can be exceed this 64K limit, but the data is still restricted to 64K.
- Other models offer different arrangements of memory space.
- Each memory model requires a separate library file, and each library files takes up space on your hard disk.

2014

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

43

Files used in C Program Development

1. Executable Files
2. Library and Runtime Files
3. Header Files
4. Programmer-Generated Files

2014

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

44

22

Files used in C Program Development

▪ Executable Files

- Executable files are stored in the subdirectory BIN. The most important executable file for the Turbo C Language is the TC.EXE.
- Executing this program places the IDE on your screen.
- The BIN directory also contains programs for the command line development process. For example
 - TCC Command line compiler
 - TLINK Command line linker
 - TCINST Customize Turbo IDE
 - TLIB Library file manager
 - MAKE File management program

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

2
0
1
4

45

Files used in C Program Development

▪ Library and Runtime Files

- Various files are combined with your programs during linking. These files contain routines for a wide variety of purposes.
- They are stored in LIB subdirectory.

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

2
0
1
4

46

23

Files used in C Program Development

▪ Library Files

- Library files are group of precompiled routines for performing specific tasks. For example, if a programmer uses a function such as **printf ()** to displayed text on the screen, the code to create the display is contained in a library file.
- A library file has a unique characteristic: only those parts of it that are necessary will be linked to a program, not the whole file.

2014

Files used in C Program Development

▪ Header Files

- The subdirectory called **INCLUDE** contains header files
- These files are text files, like the one you generate with a word processor or the Turbo C editor. Header files can be combined with your program before it is complied, in the same way that a typist can insert a standard heading in a business letter.
- Each header file has a “. h” file extension.

2014

Files used in C Program Development

- **Programmer-Generated Files**

- You can place the programs that you write in any subdirectory you choose; for instance, a subdirectory under TC.

2014

Writing a Program in C Language

- The edit window should be active, with the cursor flashing at the upper right corner. Type a program as follows:

```
void main (void)
{
    printf ( " This is my first program in C Language. " ) ;
}
```

2014

Writing a Program in C Language

- Type as you would on a typewriter. Characters will appear where the cursor is positioned.
- Press [Enter] key to move the cursor to the beginning of the next line.
- If you make a mistake, use the [Backspace] key to delete the character to the left of the cursor, or the [Del] key to delete the character under the cursor.

2014

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

51

Saving the Program in C Language

- After you have typed in the source file for the program, you should save it on your disk or hard disk. To do this, select the Save option from the File menu. You can also accomplish the same effect simply by pressing the [F2] function key.
- In C language a new file is created by a temporary name NoName00.C.
- You can save your source file as first.c or first.cpp.
- Saving the source file is not necessary before compiling and running your program, but it is a good idea and programming practice.

2014

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

52

26

Compiling and Making an .exe file

- After you have written the source file for your program, you need to turn it into an executable file. The compiler, which is a part of the IDE, translate the source code into another file known as (.obj) Object file, consisting of the machine language.
- The linker then links the entire necessary object files together to produce a final executable program with the extension (.exe).
- In Turbo C IDE, compiling and linking can be performed together in one step. There are two ways to do this: either you can select make EXE file from the compile menu, or you can press the [F9] key.

2014

Compiling and Making an .exe file

- If there is no errors in the compiling and linking process, you are ready to run the program by simply selecting Run option from the Run menu bar or by pressing the [Ctrl] [F9] key combination.
- To see the program output, select user screen option from the window menu or press [Alt] [F5].
- To exit from the Turbo C IDE, select the Quit option from the file menu.

2014

Compiler vs Interpreter

- Compiler translates the whole source program into machine language at once while the Interpreter translates the source program into machine language line by line.

2014

The Basic Structure of C Program

1. Function Definition
2. Delimiter
3. Statement Terminator
4. The printf () function

```
void main (void)
{
    printf ( " This is my first program in C Language. " ) ;
}
```

2014

The Basic Structure of C Program

■ Function Definition

- All C Language programs are divided into units called "Functions".
- A function in C will always a () at its end. Main() is always be the first function to be executed and is the one to which control is passed when the program is executed.
- The word "void" preceding the "main" specifies that the function main() will not return a value. The second "void", in the parenthesis, specifies that the function takes no arguments.

2014

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

57

The Basic Structure of C Program

■ Delimiter

- Following the function definition are the braces, which signals the beginning and ending of the body of the function.
- The opening brace " { " indicates that a block of code that forms a distinct unit is about to begin.
- The closing brace " } " terminates the block code.

2014

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

58

29

The Basic Structure of C Program

▪ Statement Terminator

- The line in our program that begins with the word " printf " is an example of a statement.
- A statement in C Language is terminated with a semicolon " ; ".
- The C Language pays no attention to any of the " White Space Character ": the carriage return (newline), the Spacebar and the Tab key.
- You can put as many or as few white spaces characters in your program as you like; since they are invisible to the compiler.

2014

The Basic Structure of C Program

▪ The printf () function

- The program line printf (" This is my first program in C Language. "); causes the phrase in quotes to be printed on the screen.
- The word printf is actually a function, just as " main " is a function name. Since " printf " is a function, therefore it is followed by parenthesis.

2014

The Basic Structure of C Program

- **The printf () function**
 - The linker looks in the CS.LIB file, finds the section of this file containing printf() and causes this section to be linked with the source program.
 - C Language distinguishes between uppercase and lowercase letters, i.e C Language is case sensitive.
 - Thus the function PRINTF() and Printf() are not the same as the function printf().

2014

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

61

Exploring the printf () Function

- The printf() function uses a unique format for printing constants and variables. For example

```
void main (void)
{
    printf ( " I am %d years old. ", 20 );
}
```
- **Output:**
I am 20 years old.

2014

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

62

31

Exploring the printf () Function

- The function printf() can be given more than one argument. For example in the above program we have taken 2 arguments in the printf() function.
- The two arguments are separated by a comma.
- The printf() function takes the vales on the right of the comma and plugs it into the string on the left.
 - Why we are using it ?
 - Why does it plug in it ?
 - Where it finds a format specifier such as %d ?

2

0

1

4

63

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

Format Specifier

- The format specifier tells the printf() where to put a value in a string and what format to use in printing the values.
 - In the previous program, the %d tells the printf() to print the value 20 as a decimal integer.
- Then what will be the method of writing a character or floating point number?
- Why not simply put the decimal integer into the original string as compared to the format specifier?

2

0

1

4

64

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

32

List of Format Specifier for printf ()

- %c single character
- %s string
- %d signed decimal integer
- %f floating point (decimal notation)
- %e floating point (exponential notation)
- %u unsigned decimal integer
- %X unsigned hexadecimal integer
- %o unsigned octal integer
- l prefix used with %d, %u, %X, %o to specify long integer. For example %ld.

2
0
1
4

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

65

Using Format Specifier in printf ()

```
void main (void)
```

```
{
    printf ( " My name is %s and I am %d years
            old. ", "Ahmad", 20 );
}
```

▪ Output:

My name is Ahmad and I am 20 years old.

2
0
1
4

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

66

33

Using Format Specifier in printf ()

```
void main (void)
```

```
{
    printf ( " The letter %c is ", ' j ' );

    printf ( " pronounced as %s ", " jay " );
}
```

▪ **Output:**

The letter j is pronounced as jay.

2
0
1
4

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

67

Using Format Specifier in printf ()

- In the above program, there are two different new things to note.
- First we have used %c for the printing of a character which is surrounded by single quotes where as %s is used for printing a string which is surrounded by double quotes.
- This is how C Language recognizes the difference between a character and a string.

2
0
1
4

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

68

34

Using Format Specifier in printf ()

- Second thing to note is that even though the output statement is printed by two separate program lines, it does not consists of two line of text on the output screen. That is because printf() does not automatically prints a new line character at the end of a line.
- So what to do for inserting a new line in a printf() statement?

2014

69

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

Class Assignment

1. Difference between Backspace and Del key.
2. Difference between obj and exe file.
3. Difference between Machine Language and High Level Language.
4. What do you mean by IDE.
5. What is the purpose of the format specifier?
6. How can we insert a new line in printf () statement?
7. Why not simply put the decimal integer into the original string as compared to the format specifier?

2014

70

CPF - Chapter No 5 : Introduction to Turbo C Programming Environment

35